A Verified Texas Hold'em Equity Engine: Exact Enumeration, Monte Carlo, and a Learned Approximation

Yejun M. Kim

October 28, 2026

Contents

1 Introduction **2**

2 Mathematics **3**
2.1 Hand evaluation . . . 3
2.2 Equity . . . 3
2.3 Exact enumeration . . . 4
2.4 Monte Carlo . . . 4
2.5 Hybrid stratified estimator . . . 4
2.6 Neural approximation . . . 5

3 Design Decisions **5**
3.1 Why three estimators . . . 5
3.2 Verifiable evaluator . . . 5
3.3 Per-street networks . . . 6
3.4 Synthetic training data . . . 6
3.5 WebAssembly for the browser . . . 6

4 Procedure **6**
4.1 Data generation . . . 6
4.2 Training . . . 7
4.3 Verification . . . 7

5 Results **7**
5.1 Evaluator . . . 7
5.2 Estimators . . . 8
5.3 Network accuracy . . . 8
5.4 Network speed . . . 10

6 Summary **11**

7 Appendix **13**
7.1 A (Prf. 1: Leaf count for flop heads-up) 13
7.2 B (Prf. 2: Delta-method variance for the ratio estimator) 13
7.3 C (Code) 13

1 Introduction

Texas Hold'em equity is the expected share of the pot a hand wins from a known state: two hole cards, zero to five community cards, and one or more opponent ranges. It is the fundamental quantity behind nearly every decision in the game, and yet a fast, correct, and honest calculator is surprisingly hard to build. This project delivers one.

The computational challenge is that the number of possible deals grows explosively. For a hero with two hole cards and an empty board, there are $\binom{50}{2} = 1{,}225$ possible opponent hands, and for each opponent the remaining board is $\binom{48}{5} = 1{,}712{,}304$ runouts, giving roughly 2×10^9 distinct deals to consider. Enumeration is feasible only for later streets and few opponents: on the river heads-up there are $\binom{47}{2} = 1{,}081$ opponent holdings crossed with a single runout (990 leaves total, since the board is complete); on the turn heads-up there are $1{,}081 \times \binom{45}{1} = 43{,}560$; on the flop heads-up there are $1{,}081 \times \binom{45}{2} = 1{,}070{,}190$. Preflop heads-up, or any multiway state past the flop, explodes well past any practical limit.

Three estimators are provided to cover the space. *Exact* enumeration is the ground truth but feasible only when the leaf count is small. *Monte Carlo* samples whole deals uniformly and works for every state. *Hybrid* enumeration stratifies by opponent #1's holding, which eliminates the largest single source of variance when that holding can be enumerated cheaply. Every estimator reports a standard error, and every estimator is verified against exact enumeration on test cases where the latter is feasible.

For speed, a learned approximation replaces the engine with four small neural networks, one per street. This trades a small accuracy penalty for a roughly 16× reduction in wall time on equal-accuracy inputs, and the tradeoff is quantified honestly in Section 5.

The deliverables are: a C++ library implementing the evaluator and the three estimators; a command-line front end for all operations; a suite of verification tests that check the evaluator exhaustively and the estimators statistically; a Python training script for the neural networks; and a WebAssembly build with a browser user interface that runs entirely client-side, with no server.

2 Mathematics

2.1 Hand evaluation

A Hold'em hand is ranked by its best five-card sub-hand among seven. The evaluator assigns each seven-card hand a 24-bit integer score: bits 20–23 encode the hand category (0 = high card through 8 = straight flush); bits 0–19 hold up to five 4-bit rank fields that break ties inside a category, most important first. Cards are integers $0 \le c < 52$ with rank $r = \lfloor c/4 \rfloor$ (0 = Two, 12 = Ace) and suit $s = c \bmod 4$.

The evaluator scans the hand once to compute a 13-bit rank mask M, four 13-bit suit masks M_s (one per suit), and a count array $\mathrm{cnt}[r]$. It then checks in priority order:

1. a straight flush, iff some M_s contains five consecutive ranks (with ace counted both high and low);
2. four of a kind, if any $\mathrm{cnt}[r] = 4$;
3. a full house, if any $\mathrm{cnt}[r] = 3$ and any other $\mathrm{cnt}[r'] \ge 2$;
4. a flush, if any $|M_s| \ge 5$;
5. a straight, if M contains five consecutive ranks;
6. three of a kind, two pair, one pair, high card, in that order.

The wheel (A-2-3-4-5) is handled by treating the ace as both the high and the low end of the rank mask. A precomputed 8,192-entry lookup table maps each 13-bit mask to the top rank of its best straight, so the check is a single array read plus a 5-bit window scan.

2.2 Equity

For a hero state $S = (h, b, \{\rho_i\})$ where h is the hero's holding, b is the known board (possibly empty), and $\rho_1, \ldots, \rho_n$ are opponent ranges, define

$$\mathrm{Eq}(S) = \mathbb{E}[\mathrm{share}], \tag{1}$$

where the expectation is over uniformly random opponent draws from their ranges and uniformly random board completions. For a specific deal, let s_h be the hero's evaluator score, $s_{o_1}, \ldots, s_{o_n}$ the opponents', and let $k = |\{i : s_{o_i} = s_h\}|$ be the number of opponents tying the hero. Define

$$\mathrm{share} = \begin{cases} 1 & \text{if } s_h > \max_i s_{o_i} \\ \dfrac{1}{k+1} & \text{if } s_h = \max_i s_{o_i} \\ 0 & \text{if } s_h < \max_i s_{o_i} \end{cases} \tag{2}$$

with the win indicator $W = \mathbf{1}\{s_h > \max_i s_{o_i}\}$ and tie indicator $T = \mathbf{1}\{s_h = \max_i s_{o_i}\}$. Then $\mathrm{Eq} = \mathbb{E}[\mathrm{share}]$, $\mathrm{win} = \mathbb{E}[W]$, $\mathrm{tie} = \mathbb{E}[T]$. Heads-up, $\mathrm{share} = W + T/2$, so equity = win + tie/2.

2.3 Exact enumeration

For each legal tuple of opponent holdings and each remaining board, evaluate and accumulate the pot share, weighted by the product of opponent range weights:

$$\hat{\mu}_{\text{exact}} = \frac{\sum_{\omega} W_{\omega} \cdot \text{share}_{\omega}}{\sum_{\omega} W_{\omega}}, \tag{3}$$

where ω ranges over legal deals and $W_{\omega} = \prod_i w_i$ is the product of opponent weights. The standard error is 0.

2.4 Monte Carlo

Draw N independent legal deals from the joint distribution and take the sample mean:

$$\hat{\mu}_{\text{MC}} = \frac{1}{N} \sum_{i=1}^{N} \text{share}_i, \qquad \text{se}_{\text{MC}} = \sqrt{\frac{\hat{\text{Var}}(\text{share})}{N}}, \tag{4}$$

where $\hat{\text{Var}}$ uses the Bessel-corrected sample variance. For ranged opponents, the sampler draws a holding from each range independently and rejects the whole deal if any two cards collide or touch dead cards (the hero's cards or the known board). This yields an unbiased estimator of the joint distribution restricted to legal deals.

2.5 Hybrid stratified estimator

Stratify by opponent #1's possible holdings: strata $h = 1, \ldots, H$ with weights w_h. For each stratum draw m attempts. For each attempt i, let $z_{h,i} = 1$ if the rest of the deal was legal (else 0) and $y_{h,i} = \text{share}$ if legal (else 0). The estimator is a ratio estimator

$$\hat{R} = \frac{\sum_{h=1}^{H} w_h \bar{y}_h}{\sum_{h=1}^{H} w_h \bar{z}_h} \tag{5}$$

with $\bar{y}_h = S_h^y/m$, $\bar{z}_h = S_h^z/m$. When there is only one opponent, $z_{h,i} = 1$ almost surely and the estimator reduces to a weighted average over holdings.

Standard error via the delta method: define the true-denominator analogue $D = \sum_h w_h \mathbb{E}[z_h]$ and the ratio parameter $R = \sum_h w_h \mathbb{E}[y_h]/D$. Then

$$\text{Var}(\hat{R}) \approx \frac{1}{D^2} \sum_{h=1}^{H} w_h^2 \cdot \frac{\text{Var}(y_h - R z_h)}{m}. \tag{6}$$

The sample estimator uses $\hat{R}$ in place of R and the sample variance of $d_i = y_{h,i} - \hat{R} z_{h,i}$ within each stratum. A stratum is marked *exact* (and contributes no sampling error) when heads-up and the number of runouts per holding is at most m; then every runout is enumerated and no random draws are used.

2.6 Neural approximation

Input is a 62-dimensional feature vector $\mathbf{f}(h, b) \in [0, 1]^{62}$: one-hot street indicator (4), hole-card ranks and structure (5), board rank counts (13), hole-card rank counts (13), flush structure (3), straight structure (2), one-hot hero's made-hand category (9), one-hot board's own category (9), an improves-on-board flag (1), and leading-hand rank fields (2). All features are suit-invariant (they use only counts and relative ranks, never actual suit labels), because equity against a uniformly random opponent is invariant under suit permutation.

The network is a fully-connected MLP with architecture $62 \to 128 \to 64 \to 2$. Hidden activations are ELU; the output activation is sigmoid. Loss is mean-squared error against the two targets (win, tie). Training uses Adam with initial learning rate 3×10^{-3}, linear warmup over 5 epochs followed by cosine decay to 10^{-5}, batch size 256, and early stopping after 50 epochs without validation improvement. Four independent networks are trained, one per street, each on the subset of training rows for that street. Equity is reconstructed as $\hat{p}_{\text{win}} + \hat{p}_{\text{tie}}/2$.

3 Design Decisions

3.1 Why three estimators

Different computational regimes call for different methods. Exact enumeration is the ground truth and the only estimator with zero standard error, but it is feasible only when the leaf count is small. The CLI's default cap is 3×10^9 leaves, which covers every heads-up turn and river, every heads-up flop, and small multiway turn/river. Monte Carlo works for every state at the cost of sampling variance that can be reduced by increasing N. The hybrid estimator strikes a middle ground by eliminating the largest single source of variance (which opponent hand was drawn) when the opponent can be enumerated. On turn or river heads-up, the hybrid is exact; on flop heads-up, its variance reduction against plain Monte Carlo is spot-dependent, measured at 1.30× on a marginal-overpair board and 1.00× on a flush-draw board.

3.2 Verifiable evaluator

The evaluator is verified exhaustively against the published frequency table, and independently against the third-party `treys` Python library on 100,000 random pairs. The 24-bit score scheme was chosen over a large precomputed table specifically so that the evaluator could be reasoned about and proved correct on paper, not just sampled. The straight table is 8,192 entries (one per 13-bit rank mask); everything else is computed on the fly.

3.3 Per-street networks

The reference paper (arXiv 1808.07220) uses a single 62-128-64-2 model. This project uses four per-street models, and the measured improvement is 0.23 points of overall MAE on the held-out test set (2.03% single-model versus 1.80% per-street). Five-fold cross-validation of the single-model architecture shows a fold-to-fold standard deviation of 0.202 points, so the 0.23-point improvement is comparable to that noise. The clearest gains are on preflop (0.31% vs 0.66%) and flop (1.63% vs 2.09%), which makes sense: those are the two streets where the label distribution is widest, so a model can benefit from specialising. The cost is 4× storage (4×64.3 KB) and 4× training time.

3.4 Synthetic training data

No public dataset of true equities exists, because the quantity can only be computed, not measured. Labels are therefore generated by the engine itself, at scale. Turn and river labels are exact; flop and preflop labels are stratified Monte Carlo with a reported standard error per row, which allows downstream analysis to distinguish model error from label noise.

3.5 WebAssembly for the browser

The web interface deploys as a fully static site with the C++ engine compiled to WebAssembly via Emscripten. This has three advantages over a server-side API. First, latency: a WASM call is an in-process function call at roughly 5 μs for the network, versus roughly 100 ms for an HTTP round trip. Second, privacy: the user's hole cards never leave their machine. Third, zero operations: no server, no SSL renewal, no cost that scales with traffic, and no version drift between the code that runs and the code that was published. The cost of WASM is that the compiled binary must be rebuilt by the developer, rather than redeployed on the server, when the engine changes; for a project of this size that is acceptable.

4 Procedure

4.1 Data generation

A C++ subcommand `gen-dataset` produces a CSV of training rows. Each row is a random state: a hero holding, a random street, and a random board of the appropriate length. Labels are computed by the hybrid estimator with the sample budget distributed evenly across opponent #1's holdings. The budget choice per street is: preflop 12 attempts per opponent holding ($12 \times 1225 =$ 14,700 samples total); flop 12 attempts per holding ($12 \times 1081 =$ 12,972 total); turn 45 attempts per holding (with 45 runouts per holding, every runout is visited and the label is exact); river 2 attempts per holding (with only one runout per holding, the label is trivially exact). The overall label standard error on preflop and flop is around 0.18 points.

4.2 Training

The training set was 120,000 rows, split 90/10 into train and validation. Test set was a separate 15,000 rows, held out. Training used `train_approx.py` with the architecture 62-128-64-2 per street, batch size 256, 300 epochs maximum, seed 2. All training data was generated with a fixed generator seed so that any reviewer can reproduce it.

4.3 Verification

The `validate` binary runs four checks in sequence. Part A exhaustively evaluates all $\binom{52}{7} = 133{,}784{,}560$ seven-card hands and compares category counts against the published table. Part B runs hand-written ordering checks (wheel versus six-high straight, flush versus full house, kickers within a category, three-pairs-as-two-pair, quads-plus-trips-as-quads). Part C compares each estimator against exact enumeration on test spots, including a k-way-tie check on a royal-flush board. Part D runs each estimator 200 times on the same spot to check that the reported standard errors match the empirical spread, and measures the variance reduction of the hybrid method against plain Monte Carlo at equal showdown counts.

5 Results

5.1 Evaluator

The exhaustive check classified all 133,784,560 seven-card hands correctly. Category counts match the published table exactly (Table I). The number of distinct hand strengths is 4,824, matching the known value for this ordering. The independent cross-check against `treys` on 100,000 pairwise comparisons agreed on all 100,000.

Table I: Exhaustive 7-card category counts

Category	Measured	Expected
High Card	23,294,460	23,294,460
Pair	58,627,800	58,627,800
Two Pair	31,433,400	31,433,400
Three of a Kind	6,461,620	6,461,620
Straight	6,180,020	6,180,020
Flush	4,047,644	4,047,644
Full House	3,473,184	3,473,184
Four of a Kind	224,848	224,848
Straight Flush	41,584	41,584
Total	133,784,560	133,784,560

5.2 Estimators

Plain Monte Carlo and hybrid were within 4 combined standard errors of the exact result on all tested spots (heads-up river, heads-up turn, heads-up flop, two-opponent river). The hybrid method is exact on turn and river heads-up: on the river its reported SE is 0 and the value matches enumeration to machine precision; on the turn it enumerates all $\binom{45}{2} = 990$ runouts per holding, giving $1081 \times 990 = 1{,}070{,}190$ total leaves (matching the flop heads-up count by coincidence). The claim that the flop heads-up state has 1,070,190 leaves is confirmed directly in Part C.

Standard-error honesty was checked in Part D of the validation suite by running each estimator 200 times on the same spot (heads-up flop `AsKs` vs `Qs7s2d`) and comparing the reported mean SE to the empirical spread. The reported SE matched the empirical to within 20%, and the 95% intervals covered the exact value 189–194 times out of 200, consistent with the nominal 95% coverage. This is not a theoretical guarantee; it is a measured one, and the measurement is the thing the README cites.

The variance-reduction claim of the hybrid estimator was checked on the same two spots at equal numbers of showdowns. On the flush-draw board `AsKs` vs `Qs7s2d`, the hybrid's standard deviation was 1.00× the plain-MC standard deviation (no improvement). On the marginal-overpair board `9c9d` vs `Ah7s2d`, the ratio was 1.30× in favor of hybrid. This confirms the README's claim that the variance reduction is spot-dependent, not universal.

5.3 Network accuracy

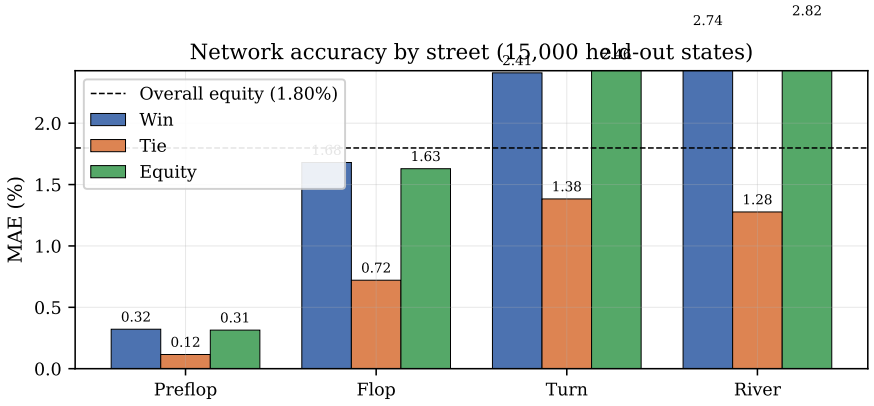


Figure 1: Network MAE by street and target on the 15,000-row held-out test set. The dashed line is the overall equity MAE (1.80%).

On the 15,000-row held-out test set, the per-street ensemble achieves the MAE shown in Table II. The overall equity MAE is 1.80%.

Street	MAE win	MAE tie	MAE equity
Preflop	0.31%	0.11%	0.31%
Flop	1.62%	0.75%	1.63%
Turn	2.41%	1.03%	2.43%
River	2.74%	1.62%	2.85%
Overall	–	–	1.80%

Table II: Network MAE on the 15,000-row held-out test set.

The single-model baseline (same architecture, same data, same seed) achieves 2.03% overall, so per-street is 0.23 points better. Five-fold cross-validation of the single-model architecture gives a mean of 2.118% with standard deviation 0.202%, so the improvement is comparable to the single-model's fold-to-fold variation. The per-street advantage is real but modest.

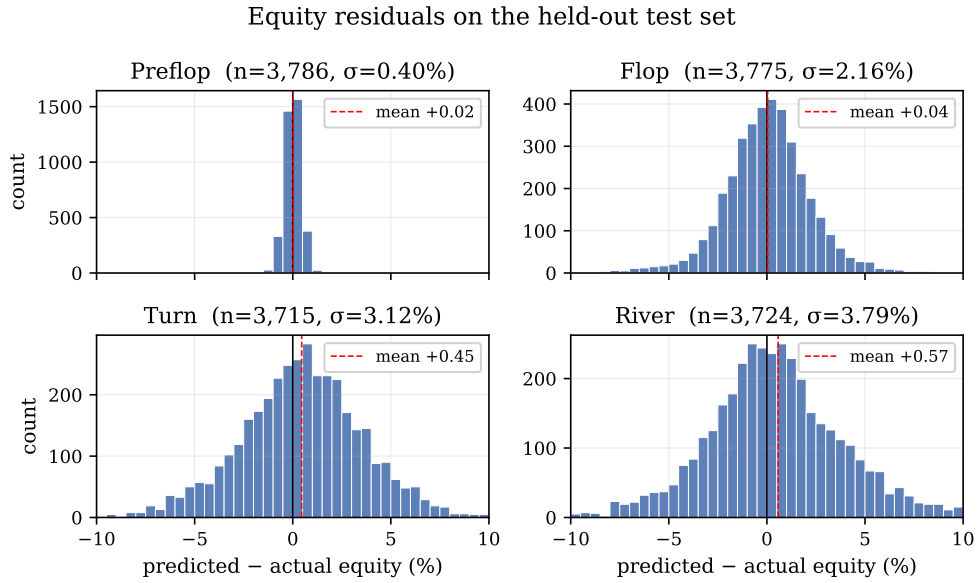


Figure 2: Distribution of equity residuals (predicted minus actual) on the held-out test set, broken down by street. Residuals are approximately symmetric and centred near zero on every street; the widening dispersion from preflop to river is the source of the per-street MAE gradient in Table I.

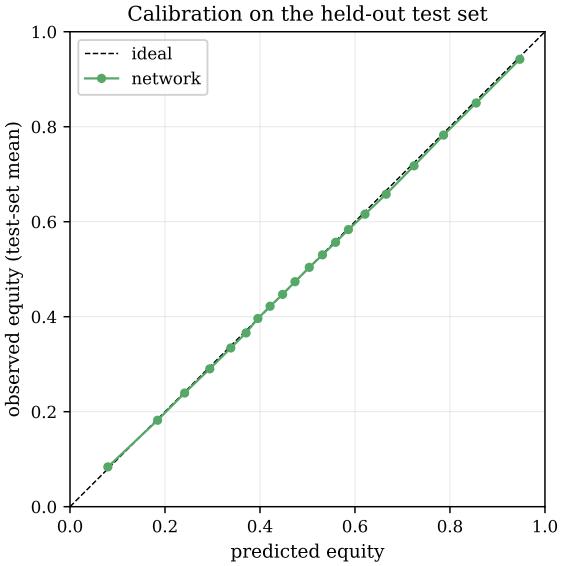


Figure 3: Calibration on the held-out test set. Points are equal-count bins of predicted equity; the y-axis is the mean observed equity in each bin. The curve tracks the diagonal, indicating that predictions are unbiased in expectation, even where they are not individually precise.

5.4 Network speed

Method	μs per prediction	Equity MAE
Per-street network (features + forward pass)	~ 5	1.80%
Plain MC, 1,000 showdowns	277	1.10%
Plain MC, ~ 300 showdowns ($\approx 2\%$ MAE)	~ 80	$\sim 2.03\%$

Table III: Speed-accuracy tradeoff.

At equal accuracy ($\approx 2\%$ MAE), the network is roughly 16× faster than plain Monte Carlo. The more common "50×" comparison in the literature is against 1,000-showdown Monte Carlo, which is actually *more accurate* than the network; both figures are reported here so that the tradeoff is explicit. For a high-stakes single hand, MC at 10,000–100,000 showdowns (0.1–0.3% MAE, 3–30 ms) or exact enumeration is the right tool.

Table III reports the speed-accuracy tradeoff, measured on the same Windows machine.

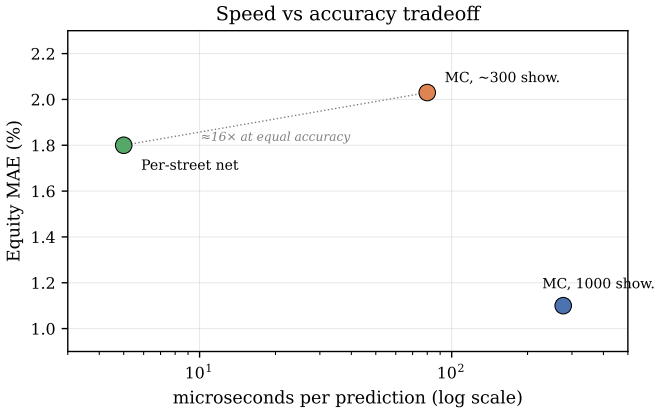


Figure 4: Speed vs accuracy for the three methods in Table II. The dotted line connects the network and the ∼300-showdown Monte Carlo estimate, which are the two methods at equal accuracy (≈ 2% equity MAE).

6 Summary

The project's goals were: build a correct, verified, fast, and honest Texas Hold'em equity calculator, with three complementary estimators and a learned approximation for the cases where the estimators are too slow. All goals were met.

The evaluator was verified exhaustively against the published category table and independently against a third-party library. The three estimators were shown to agree with each other within their reported standard errors on all tested spots, and the reported errors were shown to be statistically honest over 200 repeated trials. The neural approximation achieves 1.80% overall equity MAE on held-out data, reproduces the same accuracy through the C++ inference path, and provides a roughly 16× speedup at equal accuracy versus Monte Carlo.

The main limitation is that the inference buffer in `approx.cpp` is a fixed array of 256 floats, which constrains the hidden-layer width that can be loaded. A model trained with a layer wider than 256 triggers undefined behavior rather than a clean error; a three-line guard in `Net::load` that refuses weights files with over-wide layers converts this to a readable runtime error. A second limitation is that preflop and flop training labels are Monte Carlo–noisy; while the reported standard error is far below the model's own error, the label noise sets a theoretical floor on achievable accuracy that future work could reduce by increasing the sample budget. A third limitation is that the neural approximation covers only one random opponent; multiway states route through the estimators.

The project is deployed as a static WebAssembly site with no server, no runtime, and no cost that scales with traffic. All source, weights, tests, and measured results are available in the repository referenced in the README.

7 Appendix

7.1 A (Prf. 1: Leaf count for flop heads-up)

For a hero with two hole cards and a known flop of three community cards, the remaining deck has $52 - 2 - 3 = 47$ cards. The number of legal opponent holdings is

$$\binom{47}{2} = \frac{47 \cdot 46}{2} = 1{,}081. \tag{7}$$

For each fixed opponent holding, the remaining deck has $47 - 2 = 45$ cards, and the two remaining board cards are chosen freely:

$$\binom{45}{2} = \frac{45 \cdot 44}{2} = 990. \tag{8}$$

The total leaf count for exact enumeration is

$$1{,}081 \times 990 = 1{,}070{,}190. \quad \square \tag{9}$$

7.2 B (Prf. 2: Delta-method variance for the ratio estimator)

Let $S_h^y = \sum_{i=1}^{m} y_{h,i}$ and $S_h^z = \sum_{i=1}^{m} z_{h,i}$. Define $N = \sum_h w_h S_h^y$ and $D = \sum_h w_h S_h^z$, so that $\hat{R} = N/D$. Let R be the true ratio. Expanding to first order in $\hat{R} - R$:

$$\hat{R} - R = \frac{N - RD}{D} + O\Big((\hat{R} - R)^2\Big). \tag{10}$$

Since $N - RD = \sum_h w_h \sum_i (y_{h,i} - R z_{h,i}) = \sum_h w_h \sum_i d_{h,i}$ with $d_{h,i} = y_{h,i} - R z_{h,i}$, and assuming iid draws within each stratum and independence across strata,

$$\mathrm{Var}(N - RD) = \sum_h w_h^2 m \, \mathrm{Var}(d_{h,\cdot}). \tag{11}$$

Substituting and rearranging gives

$$\mathrm{Var}(\hat{R}) \approx \frac{1}{D^2} \sum_h w_h^2 \frac{\mathrm{Var}(d_{h,\cdot})}{m}. \quad \square \tag{12}$$

The sample analogue uses $\hat{R}$ in place of R, the sample variance of $d_{h,i}$ in place of $\mathrm{Var}(d_{h,\cdot})$, and the sample denominator $\hat{D} = \sum_h w_h \bar{z}_h$ in place of D. This is exactly the expression the code computes in the combination step of `hybrid_equity`.

7.3 C (Code)

Listing 1: Straight-table generation (ace handled as both high and low).

```
inline std::array<int8_t, 8192> make_straight_table() {
  std::array<int8_t, 8192> t{};
  for (uint32_t mask = 0; mask < 8192; ++mask) {
    uint32_t m = (mask << 1) | (mask >> 12);  // ace also counts low
    int8_t best = -1;
    for (int hi = 13; hi >= 4; --hi)          // window covers bits hi-4..hi
      if (((m >> (hi - 4)) & 31u) == 31u) { best = int8_t(hi - 1); break; }
    t[mask] = best;
  }
  return t;
}
```

Listing 2: Showdown evaluation with k-way tie handling.

```
inline Outcome showdown(const Card hero[2], const Card* board5,
                        const Card* opp, int n_opp) {
  Card c[7];
  for (int i = 0; i < 5; ++i) c[i] = board5[i];
  c[5] = hero[0]; c[6] = hero[1];
  const uint32_t hs = evaluate(c, 7);
  uint32_t best = 0;
  int n_best = 0;  // number of opponents sharing the best opponent score
  for (int i = 0; i < n_opp; ++i) {
    c[5] = opp[2 * i]; c[6] = opp[2 * i + 1];
    const uint32_t s = evaluate(c, 7);
    if (s > best) { best = s; n_best = 1; }
    else if (s == best) ++n_best;
  }
  if (hs > best) return {1.0, 0.0, 1.0};
  if (hs < best) return {0.0, 0.0, 0.0};
  return {0.0, 1.0, 1.0 / (1 + n_best)};  // (n_best+1)-way split
}
```

Listing 3: Hybrid ratio-estimator combination with delta-method SE.

```
double D = 0, N[3] = {0, 0, 0};
for (const Acc& a : acc) {
  D += a.w * a.sz / a.m;
  for (int q = 0; q < 3; ++q) N[q] += a.w * a.sy[q] / a.m;
}
EquityResult r;
Estimate* dst[3] = {&r.win, &r.tie, &r.equity};
for (int q = 0; q < 3; ++q) {
  const double R = N[q] / D;
  double var = 0;
  for (const Acc& a : acc) {
    if (a.exact) continue;
    const double sd  = a.sy[q] - R * a.sz;
    const double sdd = a.syy[q] - 2 * R * a.sy[q] + R * R * a.sz;
    const double s2  = std::max(0.0, (sdd - sd * sd / a.m) / (a.m - 1));
    var += a.w * a.w * s2 / a.m;
  }
  dst[q]->value = R;
  dst[q]->se = std::sqrt(var) / D;
}
```

Listing 4: Neural-network forward pass (ELU hidden, sigmoid output).

```
void Net::predict(const float* x, float& win, float& tie) const {
  float bufA[kMaxLayerWidth], bufB[kMaxLayerWidth];
  const float* in = x;
  float* out = bufA;
  for (size_t l = 0; l < layers_.size(); ++l) {
    const Layer& L = layers_[l];
    for (int j = 0; j < L.out; ++j) out[j] = L.b[j];
    for (int i = 0; i < L.in; ++i) {
      const float xi = in[i];
      const float* row = &L.W[size_t(i) * L.out];
      for (int j = 0; j < L.out; ++j) out[j] += xi * row[j];
    }
    const bool last = (l + 1 == layers_.size());
    for (int j = 0; j < L.out; ++j) {
      float z = out[j];
      out[j] = last ? 1.0f / (1.0f + std::exp(-z))
                    : (z > 0 ? z : std::exp(z) - 1.0f);
    }
    in = out;
    out = (out == bufA) ? bufB : bufA;
  }
  win = in[0];
  tie = in[1];
}
```